Presentation Of Two Multivariate Cryptosystems Based On The Factorisation Of Polynomials

Borja Gómez

December 27, 2020

Abstract

This paper focus in the continuation of the work presented in Sept 2019 by the author REF. The scheme was broken by REF showing that it lacked from the IP assumption, thus knowing the central map plus the public key reveals the linear or affine transformation T by basic linear algebra. In this paper, two new schemes based on the factorisation ideas are presented. In addition, these schemes base their security, between others, in the IP2 assumption. A different cryptanalysis of REF is presented as a way of showing more vulnerabilities from that early design.

1 Introduction

1.1 Polynomial Multiplication As An HFE Polynomial

It's well known that the there exists an isomorphism between the ring of Linearised Polynomials over F_q and the ring of matrices $M_{n\times n}(F_p)$. This is, we can find two Linearised Polynomials $L_1(X), L_2(X)$ over F_q from the matrices M_{L_1}, M_{L_2} by linear interpolation over F_q. Before going into interpolation, let's define the morphism that sends a Linearised Polynomial to a $n \times n$ matrix over F_p and viceversa:

$$\theta : \mathcal{L}_n \mapsto M_{n\times n}$$
$$\theta^{-1} : M_{n\times n} \mapsto \mathcal{L}_n$$

Recall that in Linearised Polynomial interpolation over F_q, to compute the morphism θ^{-1}, the Linearised Polynomial from a matrix M having columns $M_1, \ldots, M_n$ is obtained. For that, turn the columns of M into polynomials $\varphi(M_1), \ldots, \varphi(M_n)$ and describe the following system of equations over F_q. Remark that the map φ sends a vector to its polynomial representation.

$$\begin{bmatrix} \varphi(e_1) & \varphi(e_1)^{q^1} & \ldots & \varphi(e_1)^{q^{n-1}} \\ \ldots & \ldots & \ldots & \ldots \\ \ldots & \ldots & \ldots & \ldots \\ \varphi(e_n) & \varphi(e_n)^{q^1} & \ldots & \varphi(e_n)^{q^{n-1}} \end{bmatrix} \cdot \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix} = \begin{bmatrix} \varphi(M_1) \\ \vdots \\ \varphi(M_n) \end{bmatrix} \qquad \alpha_i, \varphi(e_i), \varphi(M_i) \in GF(p^n)\ \forall i\ 1 \leq i \leq n$$

Then the Linearised Polynomial Interpolation problem can be formulated as: Given input canonical polynomials $\varphi(e_1), \ldots, \varphi(e_n)$ and output polynomials $\varphi(M_1), \ldots, \varphi(M_n)$ find the coefficients α_i such that the linearised polynomial satisfies $L(\varphi(e_i)) = Y_i$. From here the reader can notice that $L(\sum_{i=1}^n \varphi(e_i)) = \sum_{i=1}^n L(\varphi(e_i))$ which is a property of Linearised Polynomials. Then the column vectors $\{M_1, \ldots, M_n\}$ form the matrix M which is equivalent to $\varphi^{-1}(L(X))$, where $\varphi^{-1}(L(X)) = Y \cdot \overrightarrow{x}$ or $L(X) = \varphi(Y \cdot \overrightarrow{x})$

Then clearly, the interpolation method finds a Linearised Polynomial $L(X)$ from a matrix M_L. Another interesting result is that Linearised Permutation Polynomials are mapped to a non-singular $n \times n$ matrix. From here, deduct that the group under composition of Linearised Permutation Polynomials is isomorphic to the General Linear Group $GL_n(F_p)$.

1.2 Applications to Cryptography

A Linearised Polynomial has at least n coefficients over F_q, where $q = p^n$. Its matrix equivalent is a $n \times n$ matrix over F_p. Both take the same amount of storage in memory, which is $\log_2 p * (n^2)$ and $n * log_2 q$. After equating and simplifying, we arrive at $\log_2 p * n^2 = n^2 \log_2 p$, concluding that both take the same space complexity.

There must exist an optimization on the underlying operations to talk about applications. Despite of Linearised Polynomials and Matrices sharing the same space complexity, It results that matrix arithmetic over F_p is faster than polynomial arithmetic over F_q. This fact translates into a more efficient time complexity: when building the Public Key of a MPKC scheme and subsequently, when signing a message or deciphering a ciphertext.

Then we are in the position to build any cryptographic scheme that involves polynomial and finite field arithmetic just by using matrix arithmetic and linear algebra. Obviously, in the case of the polynomial central map $\mathcal{F}$ i.e take an HFE polynomial of degree at least D:

$$\mathcal{F}(X) = \sum_{\substack{i=0 \\ q^i+q^j \leq D}}^{n-1} \sum_{j=0}^{n-1} \alpha_{i,j} X^{q^i+q^j} + \sum_{i=0}^{n-1} \beta_i X^{q^i} + \gamma$$

We can compute this HFE polynomial using matrix arithmetic and performing fewer polynomial multiplications. Define the Linearised Monomial Group χ, which is a matrix group with n matrix elements $\{\theta^{-1}(X^{q^0}), \ldots, \theta^{-1}(X^{q^{n-1}})\}$ isomorphic to C_n. This group can be easily precomputed for any finite field. The finite field $GF(p^n)$ is isomorphic to a matrix group $\mathcal{M}$ having $p^n - 1$ elements, under multiplication. This is a result of the following linear representation of a polynomial in $GF(p^n)$ into a $n \times n$ matrix over F_p:

$$p(x) = \sum_{i=0}^{n-1} a_i x^i, q(x) = \sum_{i=0}^{n-1} b_i x^i$$

$$p(x) * q(x) = \sum_{i=0}^{n-1} p(x) * b_i x^i = (p(x) * x^i) * b_i \equiv_{f(x)} r(x)$$

$$M_{p(x)} = \{\varphi(p(x)) \equiv_{f(x)}, \ldots, \varphi(p(x) * x^{n-1}) \equiv_{f(x)}\}$$

$$\varphi(M_{p(x)} \cdot (b_0, \ldots, b_{n-1})) = p(x) * q(x) \equiv r(x) \pmod{f(x)}$$

Morover, the matrix $M_{p(x)}$ can be precomputed for any finite field where $f(x)$ is an irreducible polynomial of degree n. Then given $p(x), q(x)$ it only suffices to substitute coefficient values a_i, b_i into this system of equations to obtain their multiplication. An interesting and important result is that the matrix $M_{p(x)}$ is always non-singular, computing its inverse gives $p(x)^{-1}$ in the finite field. We can also find it's characteristic polynomial and eigendecomposition over F_p.

Continuing with the theory, the Direct Product $\mathcal{M} \times \chi$ gives a group where the elements are the pairs $(M_{p(x)}, M_{X^{q^a}})$ under matrix multiplication. Then every finite sum over this group is a Linearised Polynomial converted to a matrix by the map θ.

A direct performance improvement is noticed when dealing with symbolic computations, as it's better to act on a set of variables $X = (x_1, \ldots, x_n)$ by linearity, this is, using linear matrices, instead of converting the variable tuple into a polynomial of degree $n-1$ and operate under multiplication with polynomial coefficients $\alpha_i \in GF(p^n)$.

The operation $\alpha X^{q^a+q^b} = \alpha X^{q^a} X^{q^b}$ can be defined as $\varphi(M_{\alpha(x)} \cdot M_{X^{q^a}} \cdot \overrightarrow{x}) * \varphi(M_{X^{q^b}} \overrightarrow{x})$. Here, the terms are taken as matrices, where operations are done using matrix arithmetic. The last operation is the polynomial multiplication $*$ in $GF(p^n)$, so output terms are converted to a polynomial by the map φ. Consequently, instead of performing 3 multiplications in the finite field $GF(p^n)$, we have just 1 polynomial multiplication. In a typical CAS, the linear algebra version of the equation $\alpha X^{q^a} X^{q^b}$ gives quadratic monomials $x_i x_j$ and linear monomials x_i^k where $k \leq 2$. Using polynomial multiplication, we'd have a bigger k, and k, l appear in monomials s.t $x_i^k x_i^l$ thus needing an extra step to simplify k, l modulo $p-1$ when $p > 2$. When $p = 2$ the step just sets $k = 1, l = 1$ in all the linear monomials.

1.2.1 Polynomial Multiplication and Factorisation using HFE polynomials

In this section a proof to construct a representative HFE Polynomial of polynomial multiplication on $Z_p[X]$ is given. First, let $q = p^n$. At a first glance, $Z_p[X]$ is infinite and an

HFE polynomial is not, but it results that we can define an HFE polynomial that input two polynomials of degree $-1+\frac{1}{2}n$ and outputs one of degree $n-2$ in $Z_p[X]$ or in F_q. This is the criteria to follow.

Take two polynomials $X_1, X_2 \in GF(p^{n-1})$. Then their multiplication on $Z_p[X]$ and not in $GF(p^n)$ gives another polynomial $Z \in Z_q[X]$ of degree at most $2(n-1)$. Then let $\overrightarrow{x} = (x_1, \ldots, x_{n/2}, x_{n/2+1}, \ldots, x_n) \in F_q^{2n}$ be a vector representation of polynomials X_1, Y_1. The goal is to separate both polynomials and multiply them by using Linearised Polynomials over F_q. To separate both polynomials from the input vector $\overrightarrow{x}$, define singular linear transformations M_{L_1}, M_{L_2} as:

$$M_{L_1} \cdot \overrightarrow{x} = \begin{bmatrix} I & 0 \\ 0 & 0 \end{bmatrix} \cdot \overrightarrow{x} = X_1 = (x_1, \ldots, x_{n/2}, 0, \ldots, 0)$$

$$M_{L_2} \cdot \overrightarrow{x} = \begin{bmatrix} 0 & I \\ 0 & 0 \end{bmatrix} \cdot \overrightarrow{x} = X_2 = (x_{n/2+1}, \ldots, x_n, 0, \ldots, 0)$$

Use the linear interpolation method over F_q to recover polynomials L_1, L_2 from M_{L_1}, M_{L_2}. Define $\overline{F}(\overrightarrow{x}) = L_1(X) * L_2(X)$ as the alternative definition of $F(X,Y)$. Then clearly, $F(X,Y) = F(\phi(\overrightarrow{x})) = X * Y = \overline{F}(\overrightarrow{\varphi(x)}) = L_1(X) * L_2(X) = X * Y$

So the multiplication $L_1 * L_2(X)$ gives a polynomial that has quadratic monomials $X^{q^a+q^b}$. Normally, this polynomial is not monic and has a degree that exceeds the bounds that allow the owner to decipher by invocation of the root finding method (i.e Berlekamp's). Concluding that polynomial multiplication can be computed as an HFE polynomial, which in the case of root finding being computationally efficient, would give us also the polynomial factorisation into two polynomials X, Y.

1.2.2 An Example of Polynomial Multiplication and Factorisation Using an HFE Polynomial

Let $p = 2, n = 6$ so $q = 2^6 = 64$. Then $\overrightarrow{x} = (x_1, \ldots, x_6) \in F_2^6$. Our goal is to imitate the multiplication in $Z_p[X]$ of the function $F(X,Y) = F(\phi(\overrightarrow{x})) = (x_1 + \ldots + x_3 t^2) * (x_4 + \ldots + x_6 t^2)$ by the product of two Linearised Polynomials over F_q. Recall from theory, that we need two 6×6 singular matrices M_{L_1}, M_{L_2} over F_p that separate $\overrightarrow{x}$ into two vectors.

$$M_{L_1} = \begin{pmatrix} I & 0 \\ 0 & 0 \end{pmatrix} . \overrightarrow{x} = \begin{pmatrix} x_1 \\ \ldots \\ x_3 \\ 0 \\ \ldots \\ 0 \end{pmatrix}$$

$$M_{L_2} = \begin{pmatrix} 0 & I \\ 0 & 0 \end{pmatrix} . \overrightarrow{x} = \begin{pmatrix} x_4 \\ \ldots \\ x_6 \\ 0 \\ \ldots \\ 0 \end{pmatrix}$$

Using Linear Interpolation over F_q we can recover Linearised Polynomials L_1, L_2 from their respective matrices.

$$L_1 = (t^5+t^4+t^2+1)X^{32} + (t^3+t^2)X^{16} + (t^5+t+1)X^8 + (t^5+t^4+t^2+t)X^4 + (t^5+t^3+t^2+t)X^2 + (t+1)X$$

$$L_2 = (t^5+t^4+t^3+t)X^{32} + (t^5+t^4+1)X^{16} + t^4X^8 + (t^5+t^3+t^2+t)X^4 + (t^5+t^3+t^2+1)X^2 + (t^4+t^3)X$$

Now, compute the product $L_1(X) * L_2(X) \equiv \overline{F} \pmod{X^{p^n-1}+1}$ in the quotient ring $\frac{Z_q[X]}{X^{p^n-1}+1}$ and recover the HFE polynomial representation $\overline{F}$ of polynomial multiplication in $Z_p[X]$.

$$\begin{gathered} \overline{F} = L_1(X) * L_2(X) \equiv (t^5+t^4+t^3+t)X^{33} + (t^3+t)X^{32} + (t^5+t^4+1)X^{17} + \\ +(t^4+t^3+t^2+t+1)X^{16} + t^4X^9 + (t^5+t^4+1)X^8 + (t^5+t^3+t^2+t)X^5 + \\ +(t^4+t^3+1)X^4 + (t^5+t^3+t^2+1)X^3 + (t^5+t^3)X^2 + (t^3+t^2+1)w \pmod{X^{p^n-1}+1} \end{gathered}$$

To check that multiplication is equivalent on both maps, select a random polynomial vector $\overrightarrow{x} = (1,1,0,0,1,1)$ which is the polynomial $1+t+t^4+t^5$. Then $F(X,Y) = F(1+t, t+t^2) = (1+t)*(t+t^2) \equiv_2 t+t^3$. On the analogue HFE polynomial the process is equivalent as $\overline{F}(X) = \overline{F}(1+t+t^4+t^5) = L_1(1+t+t^4+t^5) * L_2(1+t+t^4+t^5) = (1+t)*(t+t^2) \equiv_2 t+t^3$.

Moreover, we can find surjections for this map as root solving over F_q could give us distinct solutions for the factorisation equation. Conretely, Sage gives 4 solutions for the equation $\overline{F}(X) - (t+t^3) = 0$ and these are $(t^4+t^2+1, t^4+t^3+t^2+t, t^5+t^3+t, t^5+t^4+t+1)$. The logic here is to think as t^3+t having 4 (up to commutativity) distinct factorisations into a product of two polynomials in $F_2[X]$.

1.3 Computational Bounds on the HFE Factorisation Polynomial

The Berlekamp's Algorithm for univariate polynomial root solving over an extension field normally has a time complexity of $\mathcal{O}(d^3 + nd^2 \log d)$ where d is the degree of $\overline{F}$ and n the length of the extension field. In L_1 and L_2 the greatest monomial term $x^{p^{n-1}}$ appears, so the degree of $\overline{F}$ is about p^{n-1} which quickly goes out of computational scope when n increases, i.e when $p = 2, n \geq 9$.

Then we must construct a new computationally efficient method for factorising a polynomial over F_p into a product of two polynomials in $Z_p[X]$.

2 The Degree Decomposition Problem

A new Algebraic Combinatoric method that bypasses the computational bounds of the previous algorithm for factoring a polynomial into two polynomials in $Z_p[X]$ has been devised. Here, instead of invoking Berlekamp's algorithm for root finding, a factorisation is done into irreducible polynomials, where these factors form the factor list. From the factor list we obtain a degree partition of the polynomial. Solving an instance of the subset-sum problem on this degree partition will give us the ability to factor a polynomial into two polynomials having equal or distinct degree over F_p. Then present the Degree Decomposition Problem as the problem that we must solve in order to factor a polynomial into a product of two polynomials, where instances of the subset-sum are encountered. Instances in plural, as there may exist surjections (many solutions) for the factorisation into two polynomials, as seen in the example REF. In addition, solving the DDP gives us the same solutions as the HFE Factorisation Polynomial, which makes this approach more attractive as it still works with bigger parameters. In this paper a subinstance of the DDP is analysed, being this the Equal Degree Decomposition Problem (EDDP), which is easier to solve because it relies in a subproblem of the subset sum called Partition problem which is solvable by a pseudo-polynomial time dynamic programming algorithm. This "easeness" allows us to create a multivariate primitive.

2.1 Equal Degree Decomposition Problem

The EDDP can be characterised as the problem of factoring a polynomial over $F_p[X]$ into two polynomials over $F_p[X]$ having both the same degree. There are some trivialities, like having an irreducible polynomial $z(X)$, which is factored into $e_n(X) * z(X)$. Or being $z(X) = 0$. In general, these values correspond to the image of $F(X,Y) = Z$, which are addressed in REF.

Recall tha the goal is to build a cryptosystem on top of the EDDP problem Take the equation $XY = Z$ in $F_p[t]$, this is, X, Y, Z are polynomials over F_p in the univariate symbol t. Then in EDDP we have the necessary condition $deg(X) = deg(Y)$ and $deg(Z) = 2deg(X) = 2deg(Y)$.

In a common crypto environment Alice knows the polynomial Z. The desire is to factor Z into two polynomials X, Y of same degree. Proceed as follows:

- A polynomial Z is given of degree $2Deg(X) = 2Deg(Y)$.
- Factor Z into k irreducible polynomials f_i over F_p such that $Z = \prod_{i=1}^{k} f_i^{e_i}$.
- Build the partition degree $\lambda_{Deg} = (\underbrace{Deg(f_1), \ldots, Deg(f_1)}_{e_1}, \ldots, \underbrace{Deg(f_k), \ldots, Deg(f_k)}_{e_k})$ having partition length $\sum_{i=1}^{k} e_i$.

- Build the polynomial partition $\lambda_{Poly} = (\underbrace{f_1, \ldots, f_1}_{e_1}, \ldots, \underbrace{f_k, \ldots, f_k}_{e_k})$ having partition length $\sum_{i=1}^{k} e_i$.. This list is needed for the last step of the algorithm.

- Let $Deg(Z) = d$ then find two distinct sequences k_1, k_2 in λ_{Deg} that sum $\frac{1}{2}d$. This is equal to find a linear combination such that $\lambda_{Deg} \cdot \alpha = \frac{1}{2}d$. In this case, finding α_1 gives α_2 as α_2 is the $1's$ complement of α_1 since finding a sequence that sums $\frac{1}{2}]d$ in λ_{Deg} gives another sequence that sums d just by using the terms that are not being summed. This is a direct consequence of the property listed in the first point. This problem can be characterised as a subproblem of the subset-sum problem, called Partition Problem such that a sequence that sums $\frac{1}{2}d$ must be found in the degree partition λ_{Deg}. This is because λ_{Deg} sums to d and in the partition problem we aim to sum $\frac{1}{2}d$ instead of sum to d.

- To translate α_1, α_2 into X, Y we must know which irreducible polynomial f_1 correspond to each position in α_1, α_2. To solve this problem the polynomial partition λ_{Poly} is used. Then $\alpha_1' = \alpha_1 * \lambda_{Poly}$ and $\alpha_2' = \alpha_2 * \lambda_{Poly}$ where $*$ stands for the AND operation or the coordinate multiplication.

- Obtain $X = \prod_{i=1}^{|\lambda_{Poly}|} \alpha_{1i}'$ such that $\alpha_{1i}' = \begin{cases} \alpha_{1i}' \to \alpha_{1i}' \neq 0 \\ 1 \to \alpha_{1i}' = 0 \end{cases}$

- Obtain $Y = \prod_{i=1}^{|\lambda_{Poly}|} \alpha_{2i}'$ such that $\alpha_{2i}' = \begin{cases} \alpha_{2i}' \to \alpha_{2i}' \neq 0 \\ 1 \to \alpha_{2i}' = 0 \end{cases}$

- Finally, we've obtained a pair of polynomials such that $XY = Z \in F_p[t]$

Following these steps we are able to factor X into two equal degree polynomials X, Y that satisfy multiplication. This is needed as the APF scheme uses $F(X, Y) = XY$ as its central polynomial map.

2.2 Brief Example

Let $p = 2, n = 8$ and $X = t^3 + t^2 + 1,\ Y = t^3 + t$. Then $Z = XY = t^6 + t^5 + t^4 + t$ over F_2. Factor or decompose Z into a product of irreducible polynomials: $Z = t^6 + t^5 + t^4 + t = t*(1+t)^2*(1+t^2+t^3)$. Define $\lambda_{Deg} = (1, 1, 1, 3)$ and $\lambda_{Poly} = (t, 1+t, 1+t, t^3+t^2+1)$. We look for a linear combination s.t $\lambda_{Deg} \cdot \alpha_1 = -1 + \frac{1}{2}n = 3$. As $type(\lambda_{Deg}) = 1^3 3^1$ then one solution is $\alpha_1 = (1, 1, 1, 0), \alpha_2 = (0, 0, 0, 1)$. recall that α_2 is the one's complement of α_1. Following the procedure, we have $\alpha_1' = \alpha_1 * \lambda_{Poly} = (t, 1+t, 1+t, 0)$ and $\alpha_2' = \alpha_2 * \lambda_{Poly} = (0, 0, 0, t^3+t^2+1)$. This gives $X = t * (1 + t)^2 = t^3 + t$ and $Y = t^3 + t^2 + 1$ which obviously gives Z under multiplication. Check that in this case both X, Y have the same values as in the start of the example, up to commutativity.

2.3 Complexity of the Partition Problem found in EDDP

It's been established that the partition λ_{Deg} that sums to W, contains elements that sum to $W/2$ as a consequence of $W = Deg(Z) = 2Deg(X) = 2Deg(Y)$. Then, when such a sum is found, the other one summing to $W/2$ is obtained by the $1's$ complement of the vector solution. This is the key to understand that every partition λ_{Deg} has this special structure and therefore, the Partition Problem has a solution

Then, for a general instance of the Partition Problem, given a set S containing k elements, define r as the n$^{\underline{o}}$ of solutions that sum to $W/2$. It results that the partition problem has $r/2$ unique solutions, as we are interested on having a sequence that sums to W, but also, having two subsequences that sum to $W/2$.

2.4 Understanding the underlying problem

A cryptographic scheme to decipher correctly in every instance, in the case of begin surjective, must try all the unique possibilities that satisfy $P(X) = Y$. As the surjective values are obtained in the Partition Problem instance, the devised algorithm for this problem must find a valid $W/2$ sum, then check if it's the correct value given by the other party, i.e Bob.

Naturally, a common solution for the subset-sum problem is to implement a Binary-Tree structure that distinguish left and right nodes, as left-right nodes eliminate one element from the set S, however left nodes don't alter the sum but right do, as $W - S[i]$. The Binary-Tree structure derived from the Partition-Problem has a symmetry in leaf nodes depending on the binary value that they hold. If a leaf node has 10110, being a valid sum, then there's another leaf node with the value 01001 that's a valid sum too. This behaviour is due to the special structure of the Partition-Problem and the 1's complement.

Recall that tipically, the Partition-Problem can be solved in multiple ways. Giving only 1 sum solves it. A solution can be an integer tuple (a subset of S), and not a binary-tuple in correspondence with S. Then, how do we know which element was taken from each position in λ_{Deg} when two or more numbers $\lambda_i, \lambda_j \in \lambda_{Deg}$ are equal? The solution is to use binary-tuples in correspondence with the set S.

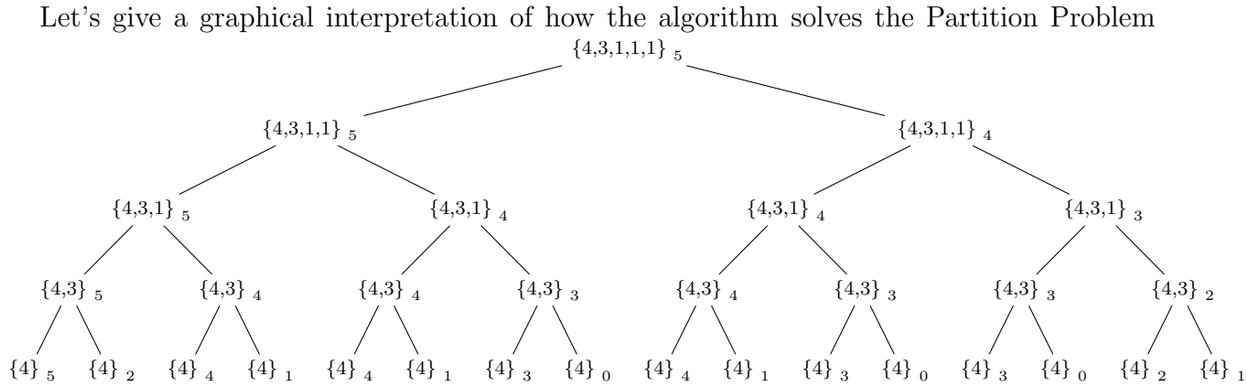
2.5 How to solve it?

Thus we are interested on solving the decisional approach, this is, true if there are subsets that sum to $W/2$ or false if not. Moreover, we're interested on tracking all the solutions found by the algorithm. This is commonly known in literature as Backtracking. It results that applying this technique, we can easiliy recover the binary-tuple associated with the elements of S. This way, we can apply the EDDP solving step to recover polynomials X, Y s.t $XY = Z$. A brief real-case example would be: given a partition with k elements, $\lambda_{Deg} = \{54, 53, 38, 32, 22, 22, 20, 7, 2, 2, 2\}$, which sums to $W = 254$ then $W/2 = 127$ so a valid sum for $W/2$ is $54 + 22 + 22 + 20 + 7 + 2 = 127$, which its binary-tuple related to S is $\alpha_1 = (1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0)$. The decryption stage would continue by obtaining α_2 by the 1's complement of α_1, continuing with the polynomial reconstruction phase, and for that we need λ_{Poly} which is not given in this example. Here, the Backtracking technique recovers both the integer tuple that sums to $W/2$ and it's positional binary-tuple related to S. Obviously, we're focused on retrieving just the binary tuple. This is why the algorithm solves the Partition Problem in a different way, as the EDDP needs a positional binary-tuple related to λ_{Deg} in order to reconstruct polynomials X, Y successfully.

2.6 An Algorithm Proposal for the Partition-Problem

An algorithm that calculates every binary-tuple related to a $W/2$ sum has been devised. It uses the typical Binary-Tree approach from the subset-sum, combined with backtracking, so everytime it finds a leaf node that satisfies the $W/2$ sum it will yield the binary-tuple α s.t $S \cdot \alpha = W/2$. The complexity of finding all the solutions in the subset-sum algorithm is $\mathcal{O}(2^n)$ being the space-complexity $\mathcal{O}(\mathcal{E} * 2^n)$ where $\mathcal{E}$ is the size of the node structure needed for the algorithm. It roughly iterates 2^n times, thus storing 2^n nodes with size $\mathcal{E}$. However, the complexity of the subset-sum algorithm doesn't apply to the Partition Problem, as symmetries exists in the binary-tuples of the leaf nodes, as the 1's complement rule would yield a solution in a different branch of the tree T. This is a clear consequence of the inherent structure of the Partition-Problem. Then, the time complexity $\mathcal{O} = 2^n$ in the subset-sum algorithm is reached on the leaf node of the right-most branch of the tree T, which the partition-problem algorithm would never reach. There's always a solution in a different leaf node having a symmetry in its binary-tuple, thus reaching it in the half time. Therefore, the worst-case complexity of this algorithm applied to the Partition-Problem is $\mathcal{O}(\frac{1}{2}2^n) = \mathcal{O}(2^{n-1})$. Computationally, interpret this result as the algorithm finding all the valid sums just by iterating half of the tree T.

Let's give a graphical interpretation of how the algorithm solves the Partition Problem



2.7 Complexity

Before explaining the complexity of solving a whole instance of the EDDP, remark that solving EDDP relies on solving the subproblem of the subset-sum problem, called the Partition Problem. The partition problem stands for: We want to retrieve valid sequences that sum to $\frac{W}{2}$, given a partition λ_k having length k, which sums to W. The time complexity of this algorithm is $\mathcal{O}(\frac{1}{2}kW)$. To continue with the complexity analysis, bounds on k, W must be defined.

In the first step of the algorithm, Z is factored into a factor list of irreducible polynomial. Factoring over F_p has a time complexity of REF. In addition, the average number of irreducible factors of a polynomial is . Then λ_{Deg} and λ_{Poly} have partition k being k the average n^{o} of irreducible factors. The partition λ_{Deg} sums to $W = d$, which tipically is $d = 2(n-1) = 126$ or 254 when $n = 128, n = 256$. Then the Partition Problem is solved in $\mathcal{O}(\frac{1}{2})kW)$ which is either.

3 Affine Polynomial Factorisation (APF)

The main difference between APF and HIP is that the central map multiplies two polynomials in $GF(p^n)$ that are transformed by S. After the transformation of S, we cannot guarantee that both polynomials are still irreducible. Both could be composite, or a combination of irreducible and composite.

Combined with the Equal Degree Decomposition Problem (EPPD) this raises a restriction based on the degree of both polynomials. It's desirable that both polynomials have the same degree after the transformation by S, so their multiplication results in $Deg(Z) = 2*Deg(X) = 2*Deg(Y)$. When deciphering, the polynomial Z is factorised into two polynomials of equal degree. It's been commented that instances of the subset-sum problem must be solved to find such pairs of polynomials of the same degree from the factor list partition extracted from Z.

3.1 Building the scheme

The cryptosystem APF consists of the following components:

- Two invertible affine transforms $T, S \in GL_n(q) \times F_q^n$.
- The map that sends a vector to its polynomial representation, $\varphi : F_q^n \mapsto F_{q^n}$ such that $\varphi(\overrightarrow{x}) \mapsto \sum_{i=1}^{n} x_i t^{i-1}$
- The polynomial central map $F(X, Y) = Y$. Different representations of this map do exist and have been already commented in REF.
- A restriction on X s.t $X = (x_1, \ldots, x_{n/2-1}, 1, x_{n/2+1}, \ldots, x_{n-1}, 1)$. Then $S(X)$ must fix both $1's$ so polynomials X', Y' have both equal degree $n-1$.
- The map that partitions a vector of even length n into two polynomials of degree $\frac{n}{2} - 1$. This is $\phi : F_q^n \mapsto \varphi(F_q^{\frac{1}{2}n}) \times \varphi(F_q^{\frac{1}{2}n})$ such that $\phi(\overrightarrow{x}) = \varphi(x_1, \ldots, x_{n/2}) \times \varphi(x_{n/2+1}, \ldots, x_n)$

Then, the public key can be defined as $P(X) = T \circ \varphi^{-1} \circ F \circ \phi \circ S(X)$

3.1.1 Structure Of the Public Key

Originally, the product $F(X, Y) = XY$ where $Deg(X) = Deg(Y) = 1 - \frac{1}{2}n$ gives $Deg(Z) = n - 2$, this is a $n-1$ tuple. Then $P(X)$ has $n-1$ polynomials in n variables. Taking into consideration the restriction made in the 4th point, $P(X)$ has $n-1$ polynomials in $n-2$ variables. Moreover, $F(X, Y) = XY$ gives an Oil-Vinegar equation, this is, the variables of X_V are mixed by themselves and with those in X_O. The affine linear action of S in the variables x_i guarantee that monomials $x_i x_j$ s.t $x_i, x_j \in X_O$ appear in $P(X)$. As a consequence,

quadratic monomials in $P(X)$ appear even when values are given for variables in X_V.

```
Input: The polynomial r(t) ∈ GF(p^n)
Output: The extended partition degree and the extended irreducible factor list
Function Factor(r(t)) is
    irrfactors ← FactorList(r(t))
    //Factorlist stores pairs (irredpoly,degree)
    //From here build both extended partitions
    factorlst ← List[]
    deglst ← List[]
    //Start from the second pair of irreducible-degree as the 1st is the identity element
    for i = 2; i ≤ Length[irrfactors]; i++ do
        poly ← irrfactors[i][1]
        deg ← irrfactors[i][2]
        for j = 1; j ≤ deg; j++ do
            deglst ← Insert(deglst, deg, Length[deglst]+1)
            factorlst ← Insert(factorlst, poly, Length[factorlst]+1)
        end
    end
    return deglst, factorlst
end
```

```
Input: The extended partition degree that sums to W = n/2 − 1, the set of values s,
       the current position pos and the backtrack structure bt
Output: Unique sequences that sum to W, covering only the left branch as the one's
        complement gives the right branch solution
Function SubsetSum(w, s, pos, bt, sols) is
    backt ← bt
    if pos == 1 then
        if w == 0 then
            Insert(backt, 0, Length[backt]+1)
            Insert(sols, Reverse[backt], Length[sols]+1)
        if w − s[1] == 0 then
            backt ← Insert(backt, 1, Length[backt]+1)
            Insert(sols, Reverse[backt], Length[sols]+1)
    backt ← Insert(backt, 0, Length[backt]+1)
    SubsetSum(w, s, pos − 1, backt)
    backt[Length[backt]] ← 1
    SubsetSum(w − s[pos], s, pos − 1, backt)
end
```